Applied Project Task 1

System Blueprint & Project Planning

Unit code: COS40007
Unit Name: Artificial Intelligence Engineering
Submission date: 15 April 2026
Studio Session No.05
Group No. CL04_G04

ACKNOWLEDGMENT OF COUNTRY

Each team member respectfully acknowledges the Wurundjeri People of the Kulin Nation, the Traditional Custodians of the land where Swinburne University of Technology stands. We recognise and honour their continuing connection to land, culture, and community, and we pay our respects to Elders past, present, and emerging.

Table of Contents

Applied Project Task 1 1
Table of Contents 3
GenAI Declaration 4
Contribution 6
Group Agreement & Sprint Plan 7
The team roles and responsibilities 7
Communication Strategy 7
On-track Behaviours 7
Off-track Behaviours 8
Conflict Resolution 8
Sprint Plan 8
System Blueprint Diagram 10
MLOps & DevOps Integration (High-Level Design) 11
Data & Training strategy 11
Retraining strategy 11
Monitoring & Drift Detection 12
DevOps Practices 12
Project Management Setup (Agile Tools) 13
User Stories sprint 1 13
Backlog & Task Breakdown 13

GenAI Declaration

We used ChatGPT to assist with improving writing clarity, defining user stories, structuring the report and explaining concepts. All content has been reviewed, edited, and validated by the team members to ensure accuracy and alignment with the assignment requirements.

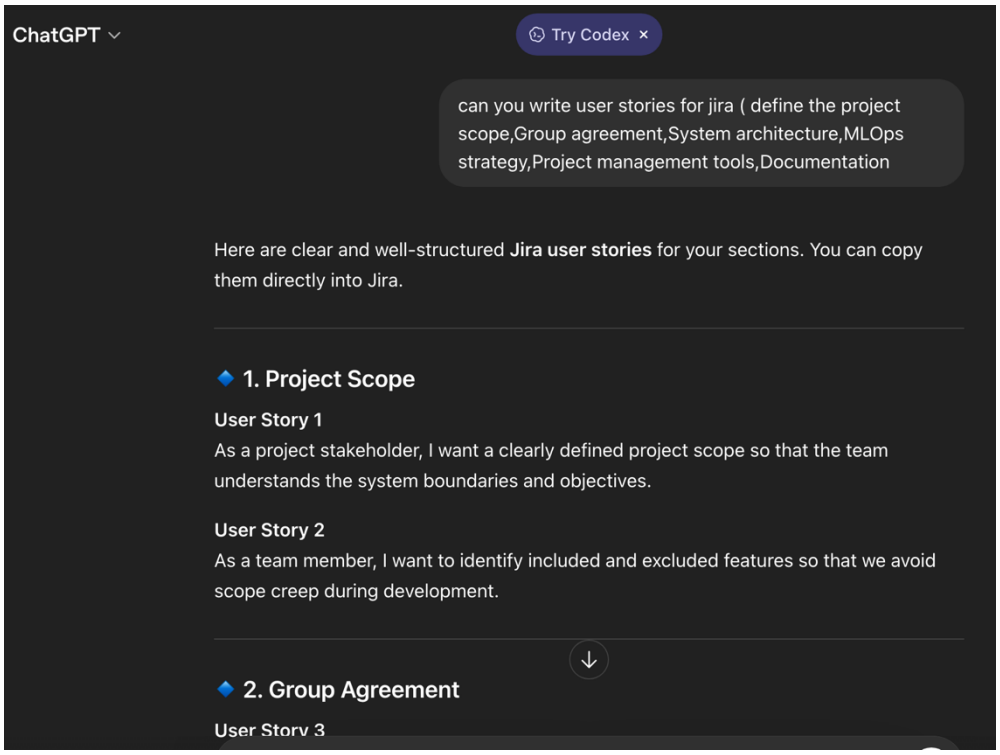

Figure 1: AI declaration - proof 1

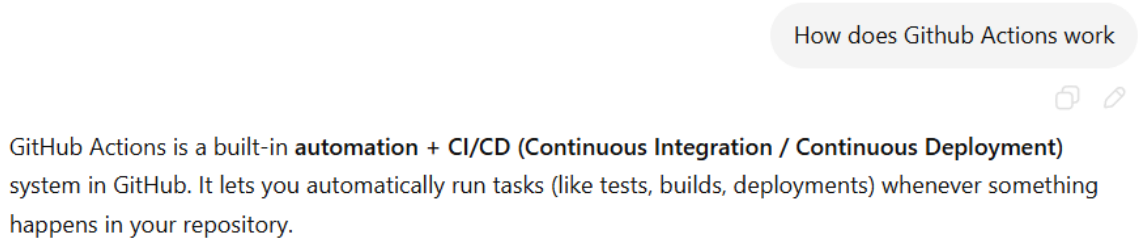

Figure 2: AI declaration - proof 2

explain why im getteing this error,
WARNING:tensorflow:6 out of the last 6 calls to <function TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data_distributed at 0x000001F5A8FCDDA0> triggered tf.function retracing. Tracing is expensive and the excessive number of tracings could be due to (1) creating @tf.function repeatedly in a loop, (2) passing tensors with different shapes, (3) passing Python objects instead of tensors. For (1), please define your @tf.function outside of the loop. For (2), @tf.function has reduce_retracing=True option that can avoid unnecessary retracing. For (3), please refer to https://www.tensorflow.org/guide/function#controlling_retracing and https://www.tensorflow.org/api_docs/python/tf/function for more details.

I'm checking the current TensorFlow guidance so I can explain the warning accurately and tie it to the most likely causes in your code.

Thought for 11s ›

That warning means TensorFlow is **rebuilding the internal computation graph over and over** instead of reusing one it already compiled. TensorFlow calls this **retracing**, and it usually happens when the same function is called with new input shapes, new Pythor ↓ ument values, or when a `@tf.function` gets recreated inside a loop. Retracing is valid, but it is slower and can hurt performance. TensorFlow +1

Figure 3: AI declaration – proof 3

Contribution

Table 1: Contribution statements

STUDENT NAME	**STUDENT ID**	CON
SOAD YUSUF	105406263	I was responsible for setting up the project management environment in Jira, including creating user stories, tasks, and sprint structure. I also developed the sprint plan and contributed to writing the Project Management Setup and system blueprint section of the report. In addition, I organised file sharing using OneDrive
GIA THANH BINH DUONG	105028560	Mainly responsible for the DevOps and MLOps and formatting, support in data research, dataset selection, and proofreading.
GAVIN RAMINDA FERNANDO	104697341	I have researched in ethical datasets to find best relevant data and manipulation of datasets to bets fit requirements but merging data, I have also developed a test bed system to test the best ML models for this project (19 models), I have also worked on the system design and the CI/CD git actions through firebase codes/deployment commands
AMIRU ADHIKARI ADHIKARI ACHCHILLAGE	105522350	Responsible for creating and maintaining the project documentation space, including the project description, system overview, and key design decisions. Assisted in reviewing and improving the MLOps planning section and conducted a final review of the full report to ensure consistency and completeness across all sections.

Group Agreement & Sprint Plan

The team roles and responsibilities

Table 2: Team roles and responsibilities

Team Roles	Student Name	Responsibilities
DevOps Engineer/ Project Manager	Soad Yusuf	Responsible for sprint planning, Jira management, and coordination. GitHub, CI/CD, and deployment setup. responsible for the submitted deliverables
UI/UX engineer	Gia Thanh Binh Duong	Responsible for UI/UX development of the system by building React (JSX) and MapBox Dashboard.
Data Engineer/ ML architect	GAVIN RAMINDA FERNANDO	Responsible for finding ethical data, data basic manipulations and engineering. prototyping relevant ML models with minimum forecasting.
Frontend Developer	Amiru Adhikari Adhikari Achchillage	Responsible for project documentation, assisting in MLOps planning, and reviewing the final report for consistency and completeness.

Communication Strategy

The team will communicate regularly to ensure smooth collaboration and progress throughout the project.

- Weekly meetings will be held via Microsoft Teams to discuss progress, challenges, and upcoming tasks
- Daily communication will take place through Discord for quick updates and coordination
- File sharing and documentation will be managed using OneDrive to ensure all members have access to the latest materials
- Task tracking and progress monitoring will be conducted using Jira

All team members are expected to respond to messages within 24 hours and actively participate in discussions and meetings.

On-track Behaviours

- Completing assigned tasks on time
- Actively participating in meetings and discussions
- Communicating clearly and respectfully with team members
- Supporting others when needed and sharing knowledge
- Keeping Jira tasks updated to reflect progress

Off-track Behaviours

- Missing deadlines or not completing assigned tasks
- Lack of communication or delayed responses
- Not attending meetings without prior notice
- Not updating task progress in Jira
- Ignoring team feedback or decisions

Conflict Resolution

- Issues are discussed openly during team meetings
- Decisions are made based on majority agreement
- Unresolved issues are escalated to the tutor if necessary

Sprint Plan

The main goal of Sprint 1 was to establish a solid foundation for the project. This sprint focused on initial setup, selecting appropriate datasets, and brainstorming to clearly define the project scope. In addition, the team defined roles and responsibilities, established a communication strategy, set up the Jira project management system, created user stories and task breakdowns, and developed a high-level system blueprint and MLOps design plan.

Table 3: Sprint plan

Sprint	Duration	Goal	Deliverables
Sprint 1	Week 5–6	Planning & Design	Project plan, Jira setup, system blueprint
Sprint 2	Week 7–9	Data Preparation/ Model Development	Clean dataset, feature selection, and Train and evaluate model
Sprint 3	Week 9–12	Dashboard integration	Dashboard + system integration
Sprint 4	Week 12–13	Testing & Finalisation	Final system testing, evaluation, report, and video presentation

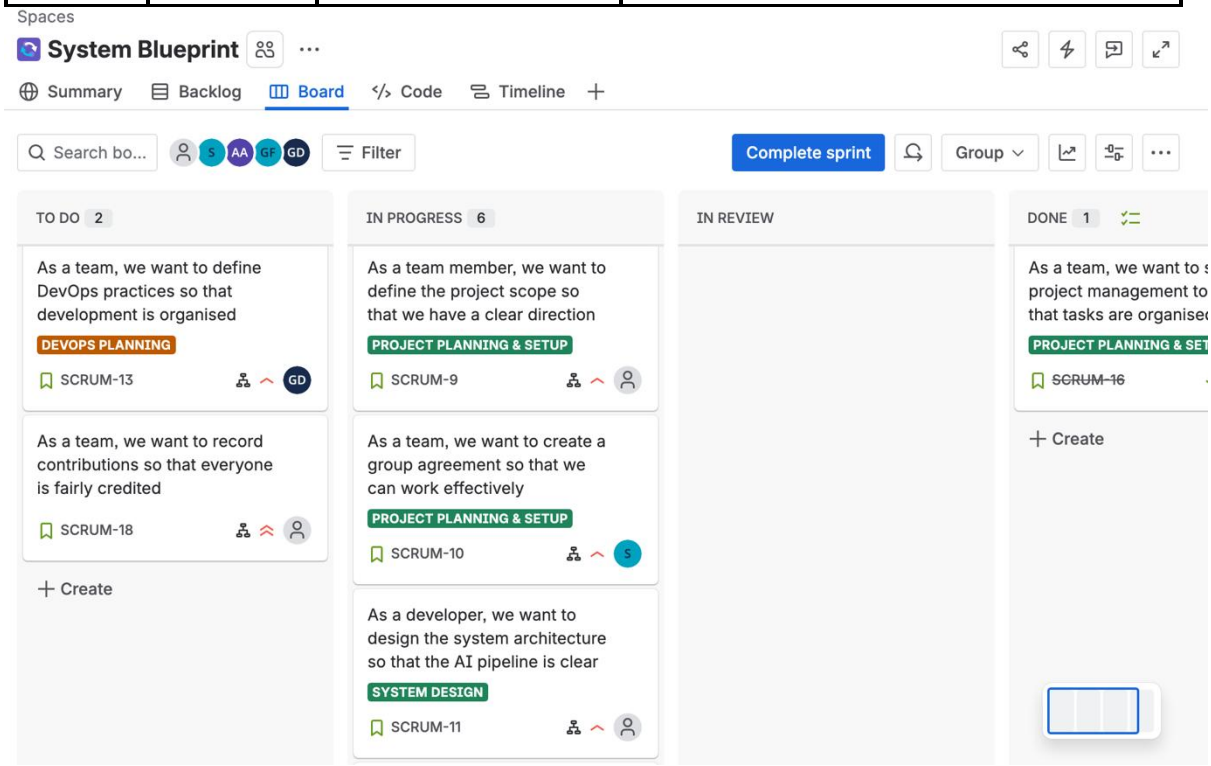

Figure 4: Sprint 1 plan board

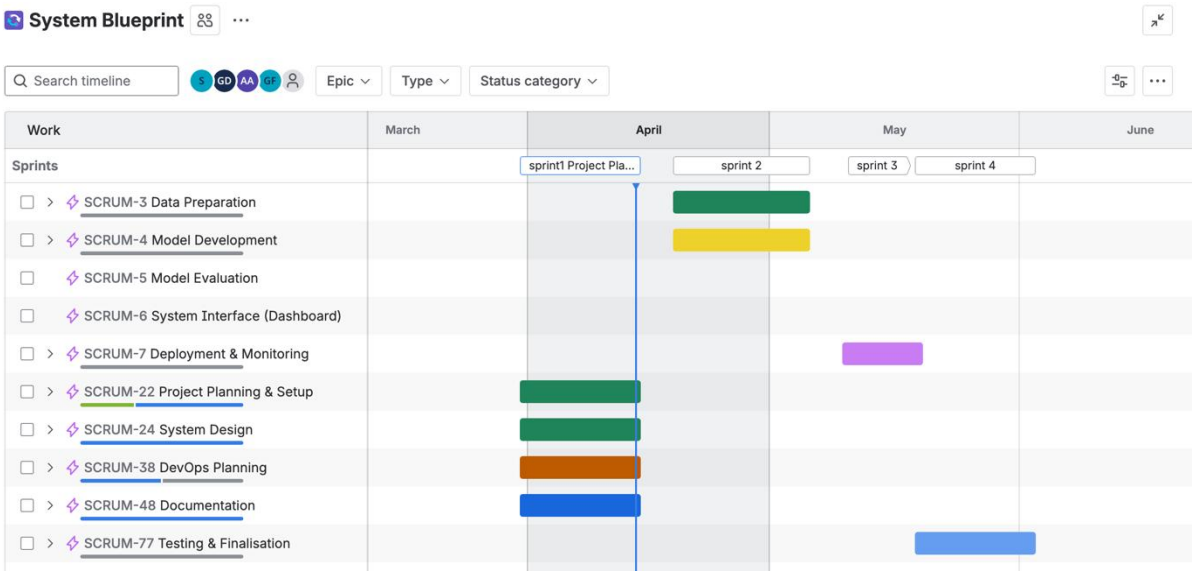

Figure 5: Sprint's timeline

System Blueprint Diagram

The system blueprint illustrates the overall architecture of the fuel usage prediction system. The process begins with data sources, including population, housing, and vehicle data, which are ingested from CSV files or APIs.

The data then undergoes preprocessing, where it is cleaned, normalised, and prepared for analysis. Feature engineering is applied to create meaningful variables such as population density, housing growth, and vehicle count. These features are used as inputs to a machine learning model, which predicts fuel usage for each suburb.

The model outputs predicted fuel usage values, which are then visualised through a Mapbox dashboard. This allows users to easily interpret and analyse fuel demand across different regions

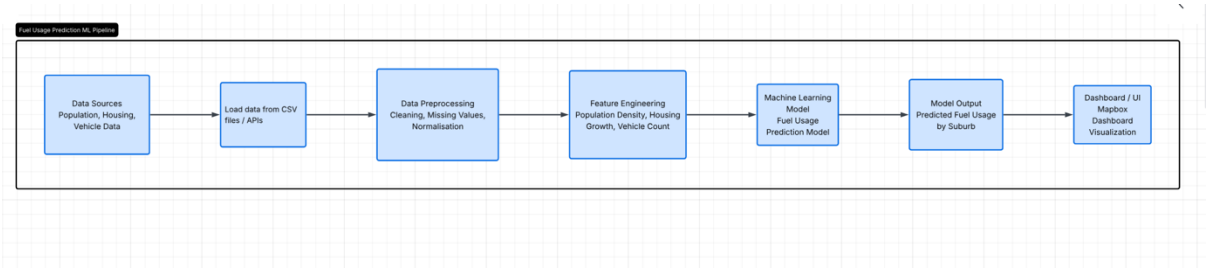

Figure 6: high-level architecture diagram 1

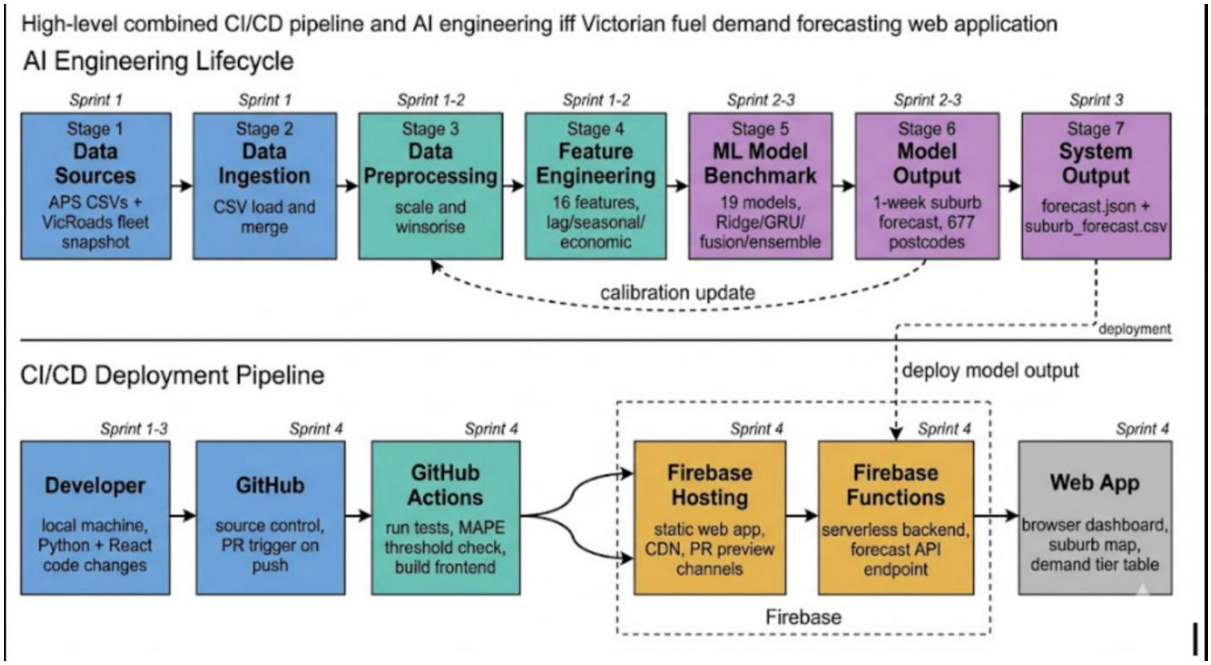

Figure 7: End-to-end architecture of the AI fuel prediction system

MLOps & DevOps Integration (High-Level Design)

This section outlines the high-level MLOps and DevOps strategy that will be applied throughout the project lifecycle. The goal is to ensure the AI system remains accurate, maintainable, and continuously integrated as new data and code changes are introduced over time. The approach covers how data will be managed for training, how and when the model will be retrained, how performance will be monitored, and how version control and deployment will be handled as a team.

Data & Training strategy

Due to the temporal nature of the data and limited sample size, we apply Time Series Cross Validation with 5 folds, ensuring that future data is never used to train on past observations. A held-out test set of 20% is reserved for final evaluation.

New data will be used for retraining because key input factors like vehicle registrations and population numbers keep changing over time. Since these changes directly affect fuel consumption, it is important for the model to learn from the latest data so that its predictions stay accurate and relevant.

Retraining strategy

Models retraining will be triggered based on time and performance based. As new data will be updated every quarter, retraining also occurs quarterly. Besides, if any figures such as RMSE, MAE or R^2's changes overpassed the determined level (for example: If RMSE increases by more than 10% compared to the baseline, or R^2 drops below 0.80, retraining will be triggered immediately.), retraining will also occur.

Monitoring & Drift Detection

Model performance will be automatically recorded into a ".csv" file after each prediction batch. The DevOps Engineer will review these logs 2–3 times per week to check whether any performance thresholds have been exceeded. If a threshold breach is detected, the team will be notified via Discord, and a decision will be made on whether to trigger immediate retraining.

Data drift will be detected using PSI (Population Stability Index) and KS test on input figures such as vehicle registrations and population data. PSI measures how much the distribution of a variable has shifted between the training data and the new incoming data, while KS test statistically compares the two distributions to determine whether they are significantly different. If either test exceeds a predefined threshold, it indicates that the input data has drifted from the data the model was originally trained on, and a retraining process will be triggered to ensure the model remains accurate.

Concept drift will be monitored by tracking the model's prediction performance over time using rolling metrics such as RMSE and MAE. Since the relationship between population, housing development, and fuel consumption may change due to external factors such as advances in technology or new government fuel policies in Australia, the model's output may no longer reflect reality even if the input data appears normal. If there is a consistent drop in prediction accuracy over time, this will be treated as a sign of concept drift, and the model will be retrained on more recent data to re-learn the updated relationship between the input and output variables.

DevOps Practices

For version control, we will use GitHub. All source code, documentation, and pipeline configurations will be managed on GitHub. The branching strategy includes three levels: a main branch for the stable, production-ready version, a dev branch for ongoing development, and individual feature branches for each team member. Each member will develop and test their work locally on their feature branch before pushing to the dev branch. Once all features are merged into dev, integration testing will be conducted. If the product passes all test cases, it will be merged into the main branch. Otherwise, it will be debugged and tested again.

GitHub Actions will be used to automate testing and deployment. Every time code is pushed to the dev branch, GitHub Actions automatically runs predefined test cases to detect errors and conflicts across the entire system — not just individual features. This ensures that issues caused by combining multiple features are caught early. If all tests pass, the pipeline will automatically deploy the latest version to Firebase, updating the MapBox dashboard without any manual effort. This removes the risk of human error during deployment and keeps the production version always up to date.

Source code versions will be managed through Git commits and tags such as v1.0.0 and v1.1.0, making it easy to track changes and roll back if needed. Machine learning model versions will be tracked using MLflow, where each model is saved with its metadata including training date, dataset used, and performance metrics such as RMSE and MAE. This

ensures that if a newly retrained model performs worse than the previous version, the team can easily roll back to the last stable model.

These strategies ensure the system remains reliable, scalable, and adaptable to real-world changes

Project Management Setup (Agile Tools)

Jira was used as the project management tool to organise and track tasks using Agile methodology. The project was structured into user stories, tasks, and subtasks, which were organised into sprints to ensure clear planning and efficient workflow management.

User Stories sprint 1

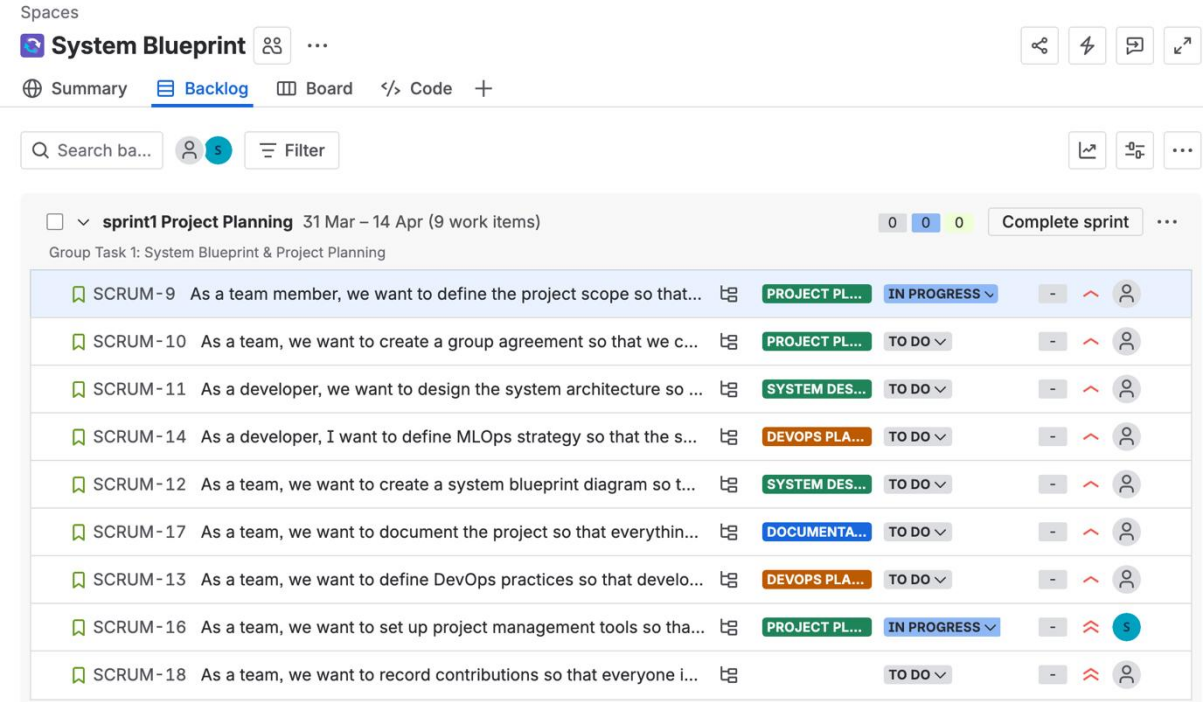

Figure 8: User stories' Sprint 1

Backlog & Task Breakdown

Each user story was further broken down into smaller, manageable tasks to support implementation. For example, the user story related to defining the project scope was divided into tasks such as defining the problem statement, selecting datasets, identifying features, and deciding the final project idea.

Tasks were assigned to team members based on their roles and responsibilities, ensuring a balanced distribution of work across the team. Priorities were set according to the importance and sequence of tasks within each sprint, allowing the team to focus on critical activities first.

The backlog was organised in Jira, where tasks were grouped under their corresponding user stories and tracked across various stages (To Do, In Progress, Done). This structure helped maintain clarity, accountability, and progress tracking throughout the project.

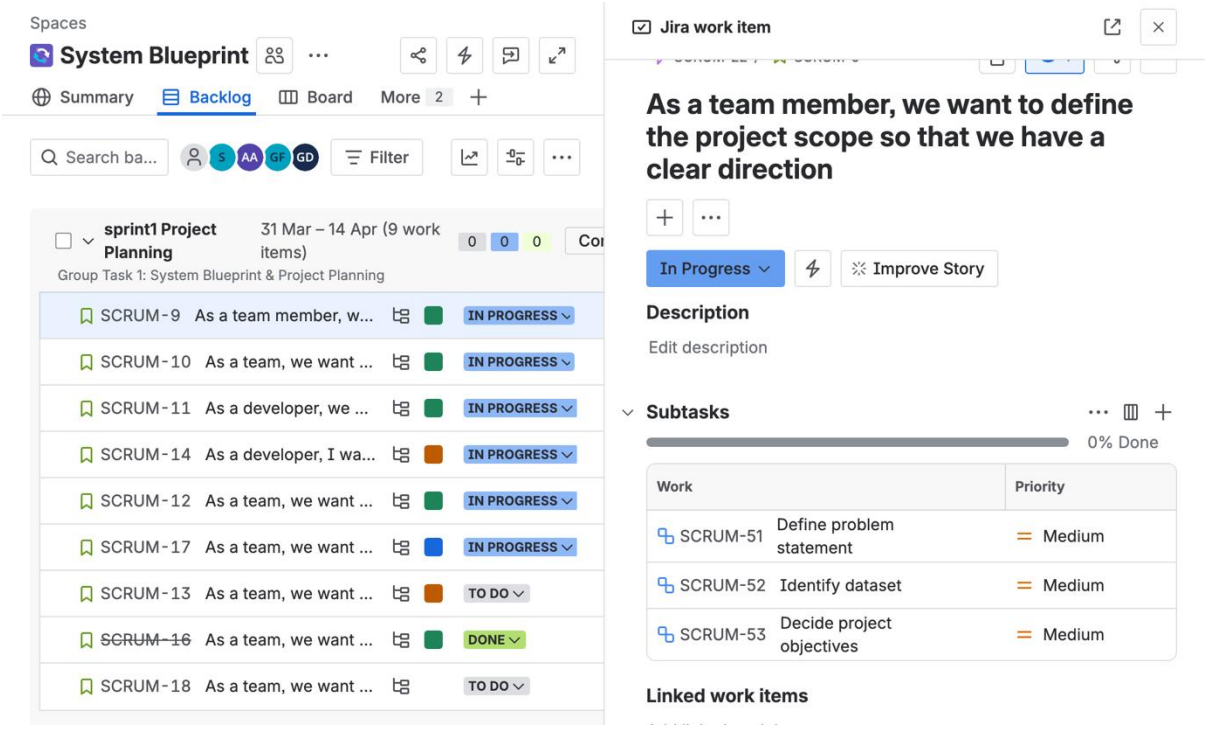

Figure 9: Sprint 1's backlog

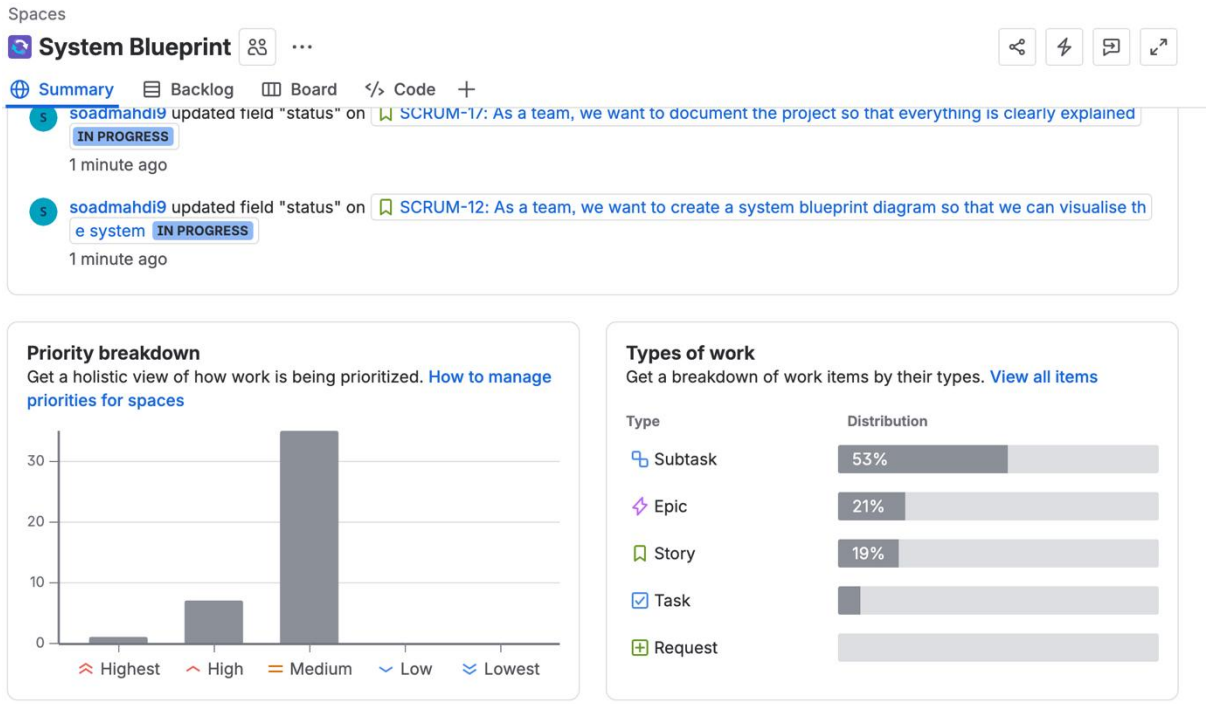

Figure 10: System summary

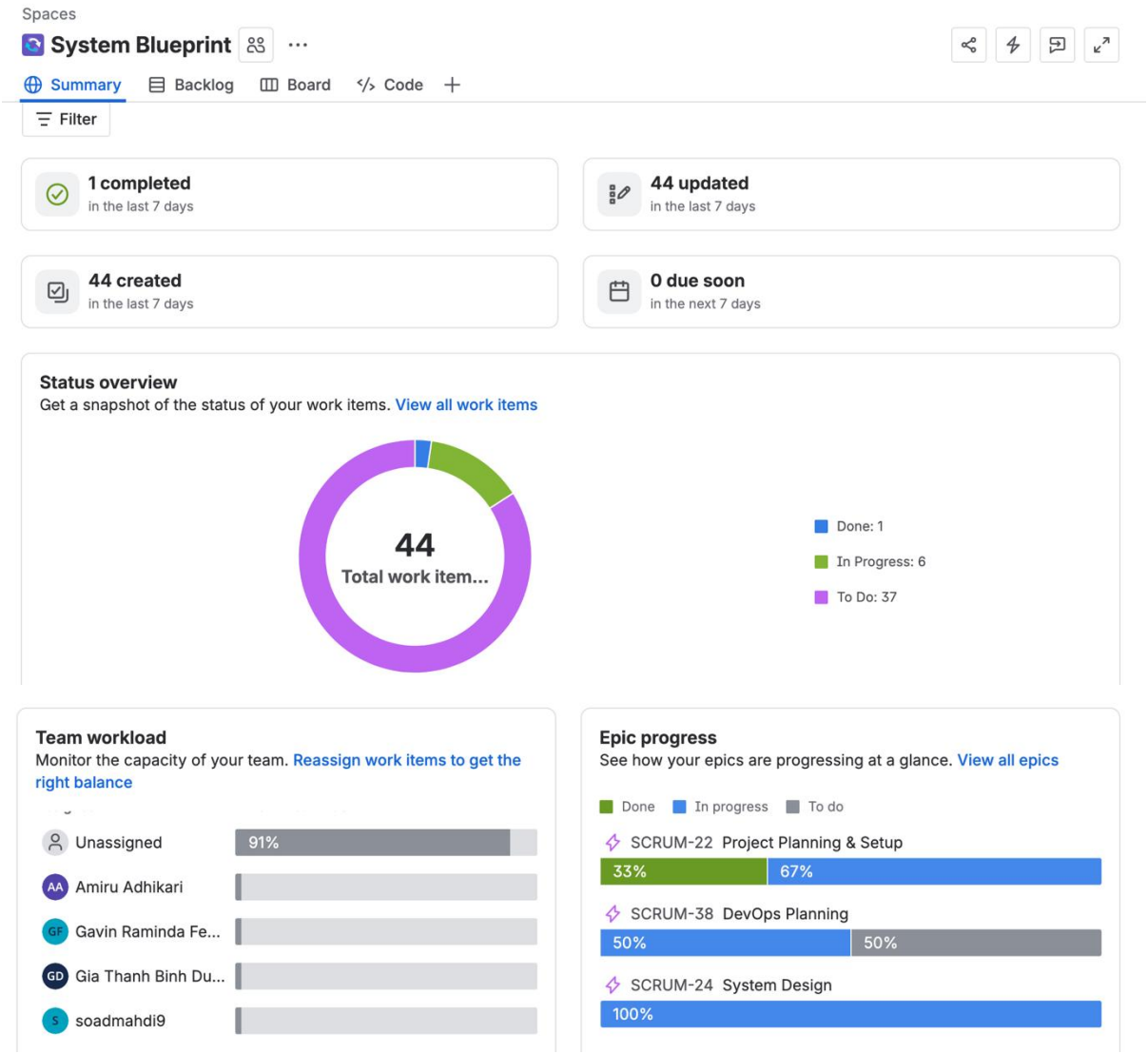

Figure 11: Sprint 1 summary